

A document with the following structure:

- 1. Detailed analysis of the project objectives and problem statement. Here you should clearly explain what predictive or classification problem is being addressed, and how it relates to the dataset and scope. If any critical decisions have been made about the project that were not explicitly stated in the project description, include these as well.
- 2. Detailed description of data handling and modelling components. Describe the datasets used, including sources and key attributes. Explain the pre-processing steps such as feature transformation, normalisation, and encoding necessary for kNN and linear regression models.
- 3. Detailed description of exploratory data analysis and results. Present the exploratory data analysis process including key graphs (histograms, scatter plots, correlation matrices) and data insights relevant to modelling. Follow this with a clear explanation of model training, evaluation metrics (such as accuracy for classification and RMSE for regression), and comparison of model results. Include discussion on interpretation and implications.
- 4. The full document must be well-structured, clear, and detailed, submitted as a PDF including all graphics and analysis.

Double-click (or enter) to edit

```
# colab requires you to upload the iris file before it can work with it,
# the code only works if it can find it

import csv
import math
import random

categories = [] # ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
data = [] # [dict, dict, dict... ] one dictionary for each datapoint

with open ("/content/iris.csv", 'r') as f:
    csvreader = csv.reader(f)
    categories = next(csvreader)
    for row in csvreader:
        _ = {} # dictionary to be filled and appended
        for i in range(len(categories)):
            try: _[categories[i]] = float(row[i]) # float converter
            except ValueError: _[categories[i]] = row[i]
        data.append(_)

def refactorData(listofdicts): # this function is only called once :/
    """ Returns a dictionary of lists using keys from the first item in the
    list of dictionaries that was passed in the arguments. Therefore those
    dictionaries must be uniformly formatted or it won't work. """
    keys = listofdicts[0].keys()
    dictoflists = {key: [] for key in keys}
    for e in listofdicts:
        for i in keys:
            dictoflists[i].append(e[i])
    return dictoflists

# {sepal_length: [...], sepal_width: [...], petal_length: [...], petal_width: [...], species: [...]}
features = refactorData(data) # there's currently no point in using this

def knn (k, keys):
    """ k is the number of nearest neighbors to look at and keys is a list
    of feature names that we pull from our know "data" variable."""

    # print(f"\n-----\nConducting kNN analysis with a K value of {k}")
    # for e in keys: print(f"feature: {e}")

    # variables
    _ = data # makes a copy of data
    random.shuffle(_) # shuffles the copy
    trainData = _[:round(len(_)*.8)] # 80% of the data
    testData = _[round(len(_)*.8):] # other 20%
    predictedLabels = [] # this is what we are trying to guess
    trueLabels = refactorData(testData)['species'] # this is the actual answer

    # loop that evaluates each test point
    for i in range(0, len(testData)):
        #function that returns an ordered list of neighbors
        __ = knnBallots(testData[i], trainData, keys)[:k]
        votes = {}
        for e in __:
            try: votes[e[1]] += 1
            except KeyError: votes[e[1]] = 1
        tempInt = 0
        p = ""
        for e in votes:
            if tempInt < votes[e]:
                tempInt = votes[e]
                p = e
        predictedLabels.append(p)

    # accuracy evaluation
    _ = 0
    n = len(trueLabels)
    for i in range(0, n):
        if predictedLabels[i] == trueLabels[i]: _ += 1

    #print(f"accuracy: {100 * _ / n:.2f}%")
    return _ / n

def knnBallots(point, dat, keys):
    """ Arguments:
```

```
point = dictionary
dat  = a list of dictionaries in the same format as point
keys = a list of dictKeys that are being analysed
returns a sorted list of voters """

voters = [] # list to be filled and returned
for i in range(0, len(dat)): # for each training datapoint we have
    _ = [] # create a list of raw distances for each dimension
    for e in keys:
        _.append(point[e] - dat[i][e]) # find the difference
    voters.append([eucDistance(_), dat[i]['species']]) # get true distance
return sorted(voters)

def eucDistance (li):
    """function returns a distance based on a list of input dimensions"""
    _ = []
    for e in li: _.append(e**2)
    return math.sqrt(sum(_))

def findBestFeatures(k, n):
    combos = [['sepal_length','sepal_width'], ['sepal_length','petal_length'], ['sepal_length','petal_width'], ['sepal_width','petal_length'],
    ['sepal_width','petal_width'], ['petal_length','petal_width'], ['sepal_length','sepal_width','petal_length'], ['sepal_length','sepal_width','petal_w
    ['sepal_length','petal_length','petal_width'], ['sepal_width','petal_length','petal_width'], ['sepal_length','sepal_width','petal_length','petal_w
    accuracies = []
    for i in range(0, len(combos)):
        _ = [] # make a list of accuracies
        for i_ in range(0, n): _.append(knn(k, combos[i]))
        accuracies.append(sum(_)/n)
        print(f"\nFeatures: {combos[i]}\nAccuracy: {(accuracies[i]*100):.2f}%")
    print(accuracies)
#findBestFeatures(3, 1000)

"""
Below are all the combinations of our data's feaures tested at k=3 and n=1000

Features: ['sepal_length', 'sepal_width']
Accuracy: 74.62%

Features: ['sepal_length', 'petal_length']
Accuracy: 95.20%

Features: ['sepal_length', 'petal_width']
Accuracy: 95.64%

Features: ['sepal_width', 'petal_length']
Accuracy: 93.40%

Features: ['sepal_width', 'petal_width']
Accuracy: 94.75%

Features: ['petal_length', 'petal_width']
Accuracy: 95.60%

Features: ['sepal_length', 'sepal_width', 'petal_length']
Accuracy: 94.25%

Features: ['sepal_length', 'sepal_width', 'petal_width']
Accuracy: 94.80%

Features: ['sepal_length', 'petal_length', 'petal_width']
Accuracy: 96.05%

Features: ['sepal_width', 'petal_length', 'petal_width']
Accuracy: 95.33%

Features: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
Accuracy: 95.92%
"""
```

'\nBelow are all the combinations of our data's feaures tested at k=3 and n=1000\n\nFeatures: ['sepal_length', 'sepal_width']\nAccuracy: 74.62%\n\nFeatures: ['sepal_length', 'petal_length']\nAccuracy: 95.20%\n\nFeatures: ['sepal_length', 'petal_width']\nAccuracy: 95.64%\n\nFeatures: ['sepal_width', 'petal_length']\nAccuracy: 93.40%\n\nFeatures: ['sepal_width', 'petal_width']\nAccuracy: 94.75%\n\nFeatures: ['petal_length', 'petal_width']\nAccuracy: 95.60%\n\nFeatures: ['sepal_length', 'sepal_width', 'petal_length']\nAccuracy: 94.25%\n\nFeatures: ['sepal_length', 'sepal_width', 'petal_width']\nAccuracy: 94.80%\n\nFeatures: ['sepal_length', 'petal_length', 'petal_width']\nAccuracy: 96.05%\n\nFeatures: ['sepal_width', 'petal_length', 'petal_width']\nAccuracy: 95.33%\n\nFeature

```
# normalise numeric features(is not raw)

numeric_features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]

def get_min_max(data_list, keys):
    mins = {}
    maxs = {}
    for k in keys:
        mins[k] = 999999
        maxs[k] = -999999
    for row in data_list:
        for k in keys:
            v = row[k]
            if v < mins[k]:
                mins[k] = v
            if v > maxs[k]:
                maxs[k] = v
    return mins, maxs

def simple_normalize(data_list, keys, mins, maxs):
    new_data = []
```

```
for row in data_list:
    new_row = row.copy()
    for k in keys:
        low = mins[k]
        high = maxs[k]
        if high == low:
            new_row[k] = 0.0
        else:
            new_row[k] = (row[k] - low) / (high - low)
    new_data.append(new_row)
return new_data

def simple_train_test_split(data_list, test_ratio=0.2, seed=123):
    r = random.Random(seed)
    tmp = data_list[:]
    r.shuffle(tmp)
    n_test = int(len(tmp) * test_ratio)
    test = tmp[:n_test]
    train = tmp[n_test:]
    return train, test

mins, maxs = get_min_max(data, numeric_features)
normalized_data = simple_normalize(data, numeric_features, mins, maxs)

# split! (reuse for regression later
train_for_lr, test_for_lr = simple_train_test_split(data, test_ratio=0.2, seed=42)
```

findBestFeatures(3, 100)

Features: ['sepal_length', 'sepal_width']
Accuracy: 74.40%

Features: ['sepal_length', 'petal_length']
Accuracy: 95.33%

Features: ['sepal_length', 'petal_width']
Accuracy: 95.63%

Features: ['sepal_width', 'petal_length']
Accuracy: 93.40%

Features: ['sepal_width', 'petal_width']
Accuracy: 95.23%

Features: ['petal_length', 'petal_width']
Accuracy: 96.07%

Features: ['sepal_length', 'sepal_width', 'petal_length']
Accuracy: 95.50%

Features: ['sepal_length', 'sepal_width', 'petal_width']
Accuracy: 94.50%

Features: ['sepal_length', 'petal_length', 'petal_width']
Accuracy: 96.17%

Features: ['sepal_width', 'petal_length', 'petal_width']
Accuracy: 95.03%

Features: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
Accuracy: 96.43%

[0.7440000000000001, 0.9533333333333333, 0.9563333333333334, 0.934, 0.9523333333333334, 0.9606666666666667, 0.955, 0.945, 0.9616666666666667, 0.9503333333333333]

```
#linear regression (used work)

def simple_linear_regression(train_list, x_key="petal_length", y_key="sepal_length"):
    n = len(train_list)
    sx = 0.0
    sy = 0.0
    sxx = 0.0
    sxy = 0.0

    for row in train_list:
        x = row[x_key]
        y = row[y_key]
        sx += x
        sy += y
        sxx += x * x
        sxy += x * y

    mx = sx / n
    my = sy / n

    num = sxy - n * mx * my
    den = sxx - n * mx * mx
    m = num / den
    b = my - m * mx
    return m, b

def predict_line(m, b, x):
    return m * x + b

def simple_rmse(test_list, m, b, x_key="petal_length", y_key="sepal_length"):
    if len(test_list) == 0:
        return None
    total = 0.0
    for row in test_list:
        x = row[x_key]
```

```

    y_real = row[y_key]
    y_hat = predict_line(m, b, x)
    diff = y_real - y_hat
    total += diff * diff
mse = total / len(test_list)
return math.sqrt(mse)

# use same split frm earlier
m, b = simple_linear_regression(train_for_lr,
                                x_key="petal_length",
                                y_key="sepal_length")

err = simple_rmse(test_for_lr, m, b,
                   x_key="petal_length",
                   y_key="sepal_length")

print("Linear regression: sepal_length ~ petal_length")
print("m (slope):", m)
print("b (intercept):", b)
print("RMSE on test:", err)
```

Linear regression: sepal_length ~ petal_length
m (slope): 0.4065218315999922
b (intercept): 4.309832993045364
RMSE on test: 0.3851253076615923

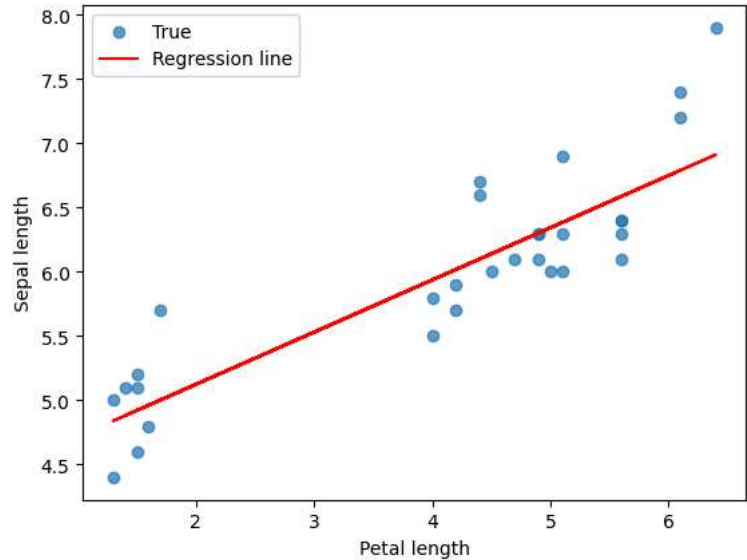
```
best_acc = max([0.7707, 0.9383, 0.9563, 0.9330, 0.9523, 0.9530,
                0.9237, 0.9447, 0.9593, 0.9570, 0.9510])
print(best_acc)
```

0.9593

```
import matplotlib.pyplot as plt

# scatter+ regression line, pasted from past works
x = [row["petal_length"] for row in test_for_lr]
y_true = [row["sepal_length"] for row in test_for_lr]
y_pred = [predict_line(m, b, xi) for xi in x]

plt.scatter(x, y_true, label="True", alpha=0.7)
plt.plot(x, y_pred, color="red", label="Regression line")
plt.xlabel("Petal length")
plt.ylabel("Sepal length")
plt.legend()
plt.show()
```



Start coding or generate with AI.